

高德 AJAX bundle 化实践总结

回首过去

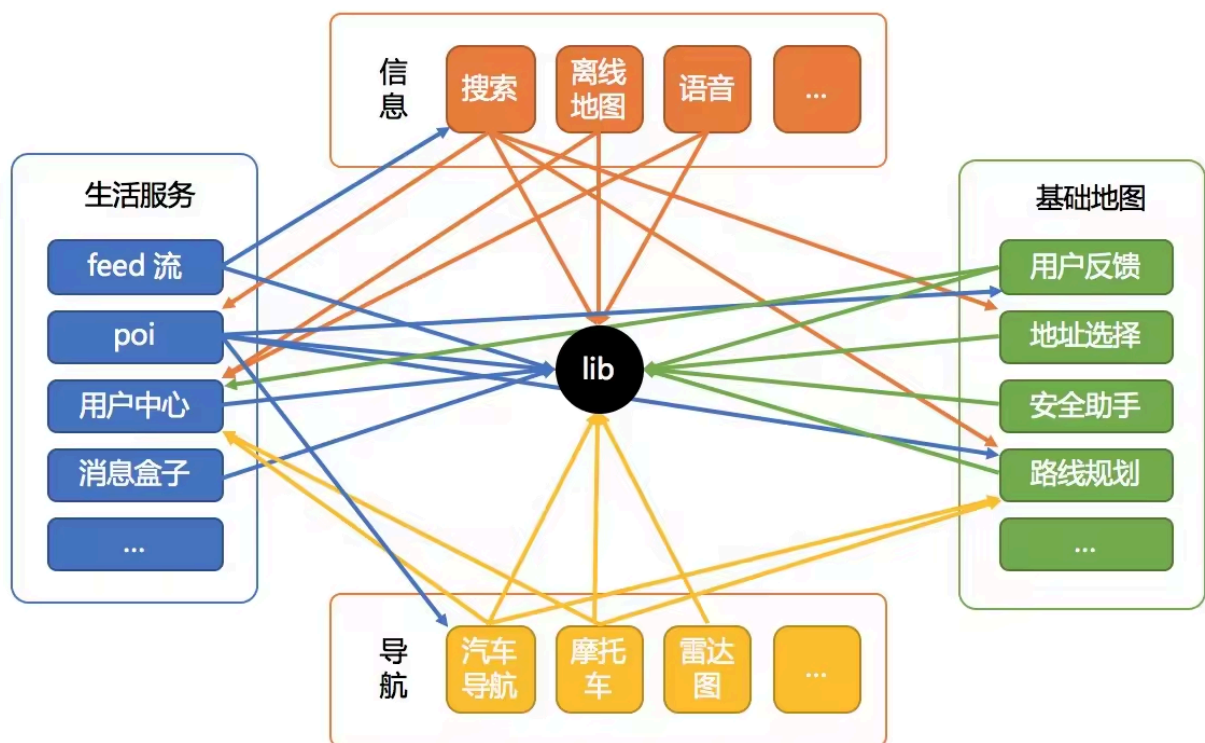
AJX 是一套基于 JS 引擎实现的移动端跨平台原生应用开发方案，实现了原生应用的高性能、可扩展，热更新。

一次开发同时可在两端生效，较大的提升了开发速度，同时可热更新的特性解决了 native 开发发布慢的问题，不断吸引新业务加入其中，推出 两年已有 60 业务模块投入使用，约 60 万行代码。

快速发展之际，也引来众多问题，快速迭代已名不符实，于是我们着手建立一套解决方案，保证业务稳定、快速迭代

AJX 前期主要聚焦于整套技术的完善，业务从零开始，划分方式比较简单粗暴，只按业务线划分了 git 仓库，子业务按文件夹存放于主仓库中。虽然划分出了公用仓库（下图中央 lib），但随着业务不断增加和迭代，各子业务之间依赖越来越多，甚至因为最终所有 AJAX 业务一起发布，业务线之间也产生了依赖

部分业务划分及依赖图：



随着依赖关系的复杂度升级，维护和测试成本成倍提升，从开发到测试、运营都苦不堪言

深挖痛点

- 开发慢：不同业务的同学修改相同的项目导致合并代码冲突多，合并代码工作量大，容易产生互相覆盖；出现问题后代码很难回滚

- 构建慢：由于所有 AJAX 业务需要一起打包为一个 `.ajax` 文件，随着代码量提升打包速度不断下降
- 测试慢：开发修改代码后无法确认影响范围，每次上线需要测试全功能回归来保证质量，浪费大量人力
- 发布慢：虽然 AJAX 业务具有动态发布能力，但各业务线开发速度不同，功能又互相依赖，最终只能按客户端的节奏整合发布，热更新的能力只能用于修复 bug

明确目标

基于这些“慢”，想要快起来最先想到的就是拆分，但拆分后怎么保证独立的编译、发布？随着业务的发展这些问题会不会再次重现？拆分时怎么保证效率与质量？显然我们需要一套新的规范来约束住业务，但约束的度怎么掌握，就像代码规范，太弱作用不大，太强又影响开发效率。通过对这些问题的思考，我们先明确了需要达到的目标，这样在改造中才能少走弯路，如下：

- 业务功能解耦，每个独立的业务功能可独立开发、编译、测试，直至上线
- 尽量减小代码变动影响范围，同时能快速确认影响范围，精准测试
- 让业务零成本或低成本切换到新架构
- 支持多业务协同发布，ABTest，用户画像，线上监控等

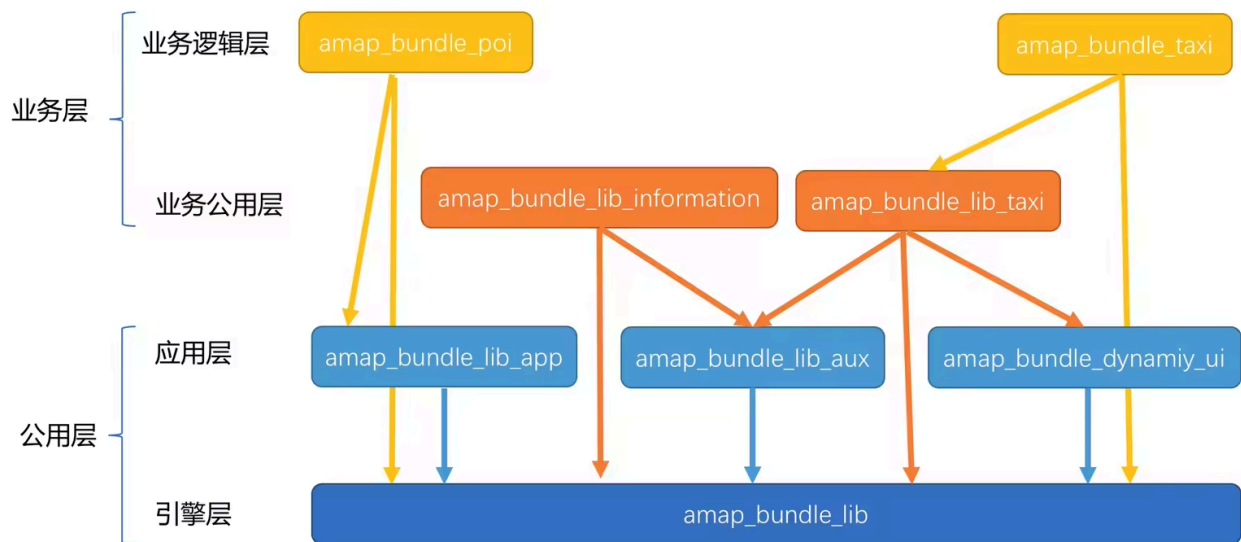
题目和答案都已确认，接下来就是解题了

平稳落地

1. 强依赖解耦

要开发编译提速，首先就要拆分代码仓库，为同时保证业务的独立运营及管理，拆分第一规则为按业务功能点进行拆分，每个独立的功能点拆分为一个代码仓库。

拆分后，bundle 之间的引用关系怎么办？按之前的每个 bundle 内分出公用部分供别的 bundle 调用？怎么把修改后的影响范围降到最低？这些问题不解决，最终还是会回到之前的情况，鉴于此，将公用功能下沉为公用 bundle。公用代码改动影响范围较大，同时对公用 bundle 进行了分层处理，按 bundle 的功能分为四层：



下层不能依赖上层，业务之间不能互相依赖，这样大大简化了依赖关系；同时越往下层，改动频率越低，减小了公用 bundle 变动影响范围

2. 弱引用管控

业务之间耦合问题已解决，业务之间的跳转、广播、服务呢？业务内部升级、重构等修改了结构，对外提供的这些功能怎么办？普通的依赖限制无法解决这个问题，于是我们定义了 protocol 的引用方式，既明确了 bundle 之间的依赖关系，又保证了 bundle 对外输出的稳定性，同时由于 bundle 内只有显示静态声明可被外部使用的功能才能被外部使用，成功限制了隐性的依赖，示例



复制代码

```
{
  "protocol": {
    // 向外暴露的功能
    "router": {
      "scheme": {
        // key 为对外提供的功能名称，value 为提供此功能的文件
        "amapscheme://search/homepage": "homepage.js"
      },
      "service": {
        // key 为对外提供的功能名称，value 为提供此功能的文件
        "amapservice://search/searchDistance": "search_distance.js"
      },
      // 其它功能
    },
    // 依赖的外部业务功能
    "dependencies": {
      "service": ["amapservice://user/login"],
    }
  }
}
```

3. 业务平稳迁移

前面已定义业务开发规范，业务需要按新的规范组织项目结构，修改文件的引用关系，查找弱引用，并声明 protocol，直接进行改造适配工作量无疑巨大，且容易出错，事倍功半，于是我们同步推出迁移工具来保证高质高效的迁移工作，主要功能如下图



有了这个工具，在业务同学配合下，仅用两周将 60 多模块、60 万代码全数迁移成功，并且新版本中迁移导致的 bug 仅为个位数

4. 工程链路支持

至此，研发问题已基本解决，最后就是如何保障快速发布与运营，更高的稳定性，创建一个完整的运维体系：工程平台来完成整个体系的运营

- 1) 全流程一体化：提测、测试、上线、监控、分析等一体化操作
- 2) 运维精细化：支持 bundle 功能维度发布，如协同发布；支持按目标用户发布，如 ABTest，用户个性化功能等
- 3) 依赖关系透明化：精确分析业务各形式依赖，确定每次发版影响范围

工程中台的建设完成，大大减小了测试范围，方便追溯所有改动，测试周期从近一个月缩短到 1 周；发布版本仅需在系统中进行简单操作确认即可，让业务可快速试错，同时线上的监控预警分析能力也将质量提升了一个档次。

展望未来

AJX bundle 化的完成标识着 AJX 已步入正轨，不再依附于客户端的背后，带动高德地图以更快的速度前行。时至今日 AJX 业务已有 80+ bundle，130 万行代码。

当然我们还不能为目前的成就而骄傲，我们的功能还要朝着智能化方向发展，下面是对未来的一些畅想

- 1) 不断变化的需求必然带来不断变化的 bundle，需要更加智能的分析来平衡但 bundle 无限扩充问题
- 2) 与小程序、H5 的共建，为业务带来更多的可能